

ML HW3

1. Model architecture

- dataset

In order to check the accuracy, I split the dataset into two parts, train and valid.
The ratio is (0.9,0.1)

- model

I try 3 different model

All models take grayscale facial images of shape (1, 48, 48), and output a 7-dim vector representing 7 emotion classes.

Simple CNN: About 2M parameters (for baseline)

Improved CNN (VGG style): About 4.7 parameters

ResNet18: About 5.3M parameters

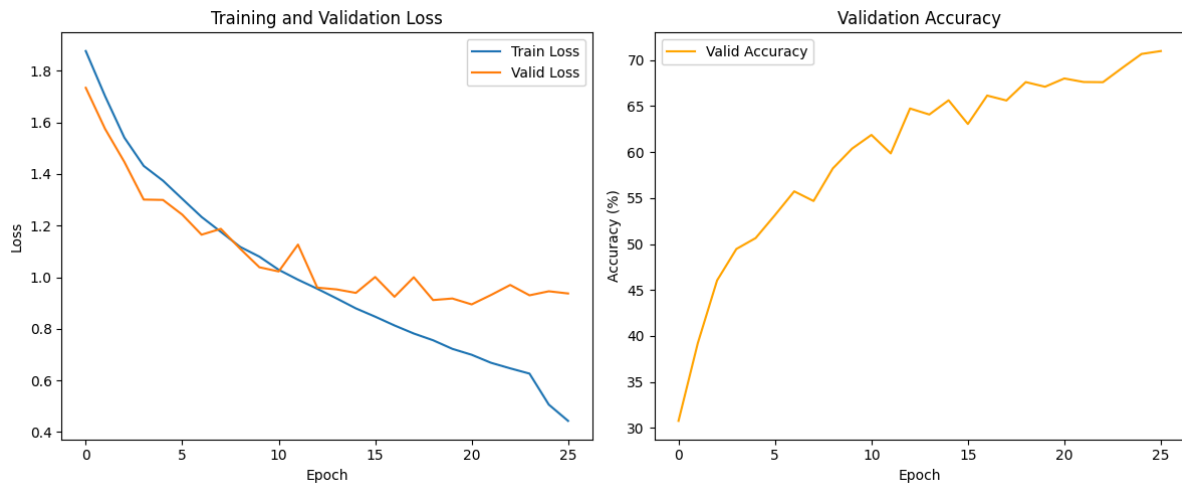
We start from a simple CNN to verify our training pipeline. Then, we adopt a deeper VGG-style architecture with Dropout and BatchNorm to enhance generalization. Finally, we implement ResNet18 from scratch to exploit residual connections for deeper network training stability and accuracy improvement. And I found that VGG-style has better performance, so I will use this model for further improvement.

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 48, 48]	640
BatchNorm2d-2	[-1, 64, 48, 48]	128
ReLU-3	[-1, 64, 48, 48]	0
Conv2d-4	[-1, 64, 48, 48]	36,928
BatchNorm2d-5	[-1, 64, 48, 48]	128
ReLU-6	[-1, 64, 48, 48]	0
MaxPool2d-7	[-1, 64, 24, 24]	0
Dropout-8	[-1, 64, 24, 24]	0
Conv2d-9	[-1, 128, 24, 24]	73,856
BatchNorm2d-10	[-1, 128, 24, 24]	256
ReLU-11	[-1, 128, 24, 24]	0
Conv2d-12	[-1, 128, 24, 24]	147,584
BatchNorm2d-13	[-1, 128, 24, 24]	256
ReLU-14	[-1, 128, 24, 24]	0
MaxPool2d-15	[-1, 128, 12, 12]	0
Dropout-16	[-1, 128, 12, 12]	0
Conv2d-17	[-1, 256, 12, 12]	295,168
BatchNorm2d-18	[-1, 256, 12, 12]	512
ReLU-19	[-1, 256, 12, 12]	0
Conv2d-20	[-1, 256, 12, 12]	590,080
BatchNorm2d-21	[-1, 256, 12, 12]	512
ReLU-22	[-1, 256, 12, 12]	0
MaxPool2d-23	[-1, 256, 6, 6]	0
Dropout-24	[-1, 256, 6, 6]	0
Conv2d-25	[-1, 512, 6, 6]	1,180,160
BatchNorm2d-26	[-1, 512, 6, 6]	1,024
ReLU-27	[-1, 512, 6, 6]	0
Conv2d-28	[-1, 512, 6, 6]	2,359,808
BatchNorm2d-29	[-1, 512, 6, 6]	1,024
ReLU-30	[-1, 512, 6, 6]	0
MaxPool2d-31	[-1, 512, 3, 3]	0
Dropout-32	[-1, 512, 3, 3]	0
AdaptiveAvgPool2d-33	[-1, 512, 1, 1]	0
Flatten-34	[-1, 512]	0
Linear-35	[-1, 128]	65,664
ReLU-36	[-1, 128]	0
Dropout-37	[-1, 128]	0
Linear-38	[-1, 7]	903
Total params: 4,754,631		
Trainable params: 4,754,631		
Non-trainable params: 0		
Input size (MB): 0.01		
Forward/backward pass size (MB): 13.72		
Params size (MB): 18.14		
Estimated Total Size (MB): 31.87		

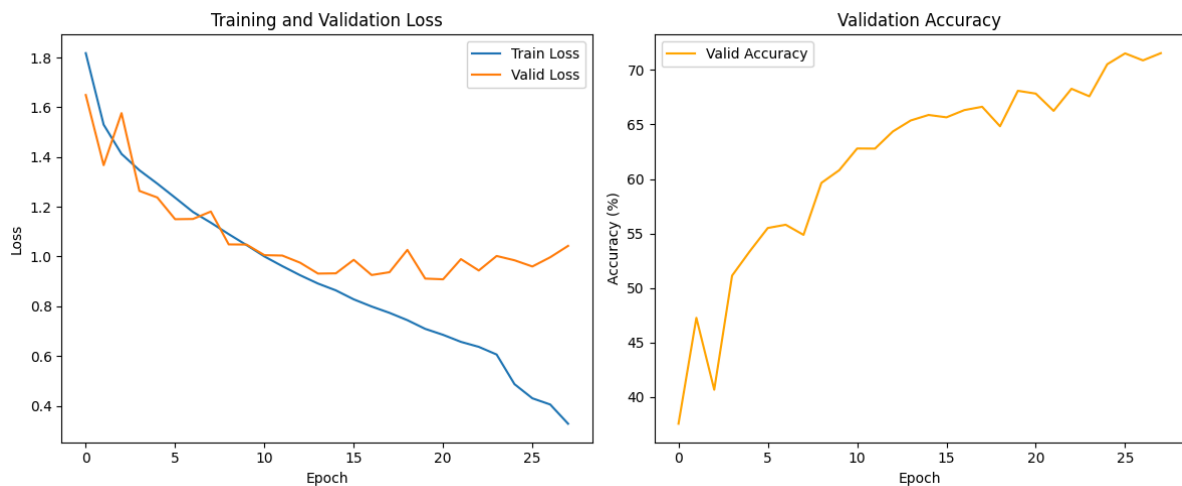
According to the output, the estimated total size is 31.87 MB, which is a lightweight model, can be run using colab's GPU.

2. Specify the loss function and plotting the loss curves

I use the Cross Entropy Loss as the loss function.

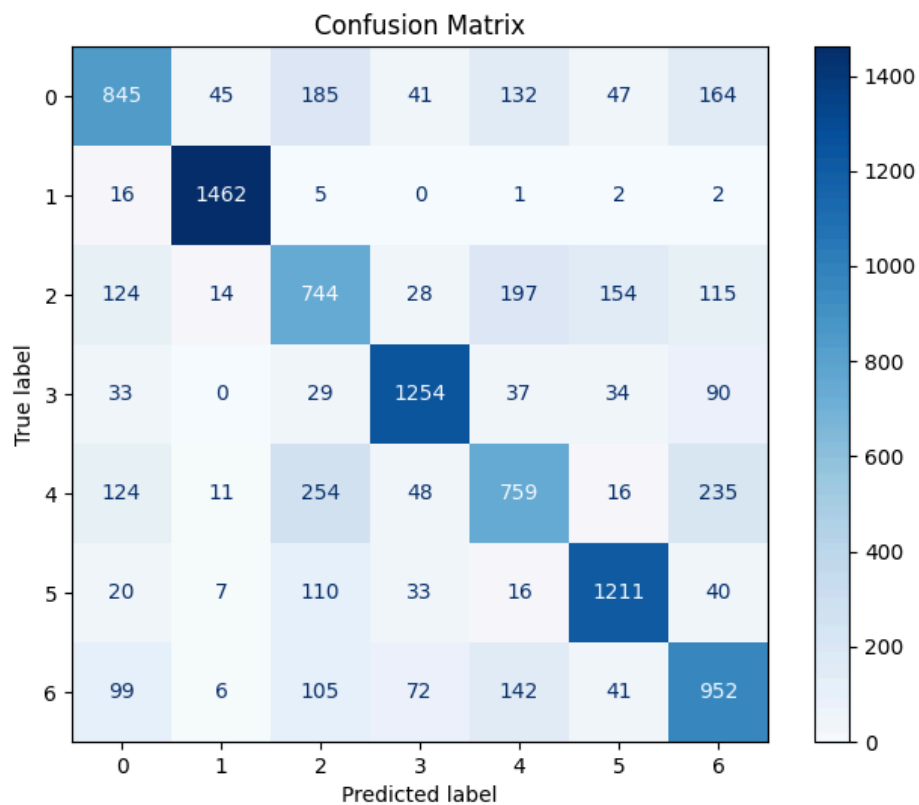


I use an early stop to prevent overfitting. As you can see, it seems that the model can be better, so I changed the patience from 5 to 7.



As you can see the valid loss is about to rise up. No more training.

Check the confusing matrix



Judging from the values of the diagonal, almost every expression has been well judged.

Classification Report:				
	precision	recall	f1-score	support
0	0.6701	0.5792	0.6213	1459
1	0.9463	0.9825	0.9641	1488
2	0.5196	0.5407	0.5299	1376
3	0.8496	0.8490	0.8493	1477
4	0.5911	0.5245	0.5558	1447
5	0.8047	0.8427	0.8232	1437
6	0.5957	0.6718	0.6315	1417
accuracy			0.7155	10101
macro avg	0.7110	0.7129	0.7107	10101
weighted avg	0.7139	0.7155	0.7135	10101

But you can see that the fear and sadness are still obviously bad, so I will use cost-sensitive learning to optimize these two.

To improve the performance, I also use **Bayesian Optimization** for Hyperparameter tuning.

parameter : [learning rate, batch size, weight decay]

In the end, I reached about 71% of the valid dataset.

3. Approach to data imbalance

```
Processing happy with label 3 | 7215 images  
Processing sad with label 4 | 4830 images  
Processing fear with label 2 | 4097 images  
Processing surprise with label 5 | 3171 images  
Processing neutral with label 6 | 4965 images  
Processing angry with label 0 | 3995 images  
Processing disgust with label 1 | 436 images
```

As you can see, I check the distribution of the train data. The data distribution is extremely uneven.

To address the issue of class imbalance in the facial expression dataset, I employed a **data-level approach** by generating additional training samples for underrepresented classes. Specifically, I used the following strategy:

A. Class Count Analysis:

I first counted the number of images in each emotion class. The class with the maximum number of images was used as the reference.

B. Data Augmentation for Minority Classes:

For classes with fewer images, I generated synthetic samples through image augmentation techniques until each class had the same number of samples as the majority class. The augmentation methods included:

- **Horizontal Flip** (`cv2.flip(img, 1)`)
- **Vertical Flip** (`cv2.flip(img, 0)`)
- **Random Rotation** within a range of ± 30 degrees

C. Saving Augmented Images:

The augmented images were saved to a new directory and their file paths were recorded along with their corresponding labels in a new CSV file. This balanced CSV was later used for training the model.

This method increases the diversity of the training data while ensuring that all emotion classes are equally represented. By balancing the dataset, I aimed to reduce the model's bias toward overrepresented classes and improve the generalization performance on minority classes such as *disgust* or *fear*.

For the training part I use cost-sensitive learning.

Cost-Sensitive Learning is a method to handle class imbalance by assigning **higher weights to underrepresented or important classes** during training.

In this case, classes like “**fear**” and “**sad**” were given more weight in the loss function. This makes the model **pay more attention to these harder classes**, improving their recall and overall performance.

It helps prevent the model from being biased toward majority classes and encourages **balanced learning** across all emotions.